PRÁCTICA 7 - CONFIGURACIONES SEGURAS Y ANÁLISIS DE SERVICIOS

Realizado por Unai García Blanco y Alberto García Martín

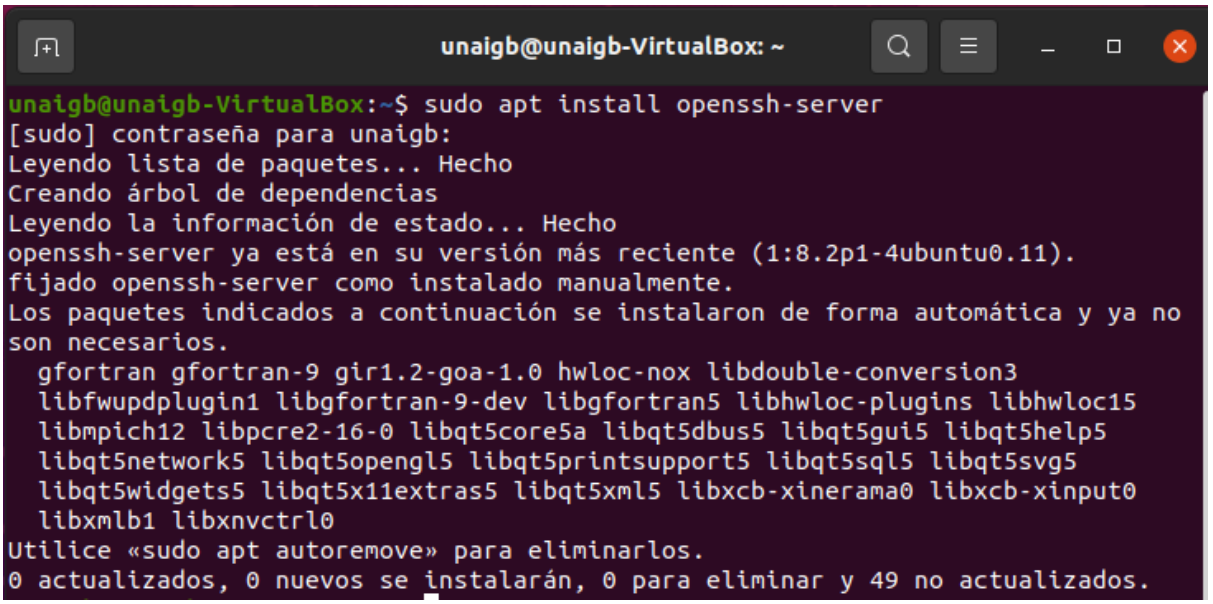
1. Configuraciones que ayudan a la seguridad....2
1.1. Servicios de ssh, ftp y https....2
SSH....2
FTP....3
HTTP....3
Pruebas....5
1.2. TCP Wrappers....6
1.3. Comparación: TCP Wrappers vs IPTables....8
2. Análisis de servicios activos....10
2.1. Análisis de servicios activos....10
2.1.1. Usar netstat para analizar servicios activos....10
2.1.2. Usar nmap para analizar puertos abiertos....12
2.2. Herramientas adicionales de análisis de puertos....14
2.2.1. Herramientas alternativas a nmap....14
2.2.2. Ejemplo práctico: Uso de Angry IP Scanner....15

1. Configuraciones que ayudan a la seguridad

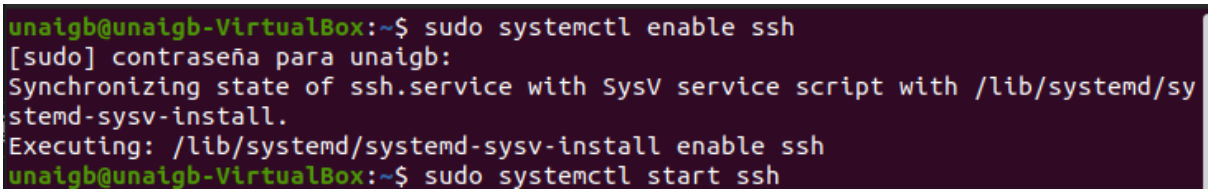
1.1. Servicios de ssh, ftp y https

Lo primero que vamos a tener que realizar es la habilitación de los servicios de conexión remota (ssh) y de subida de archivos (sftp).

SSH



En nuestro caso ya estaba instalado. A continuación lo habilitaremos e iniciaremos mediante los comandos "*systemctl enable ssh*" y "*systemctl start ssh*"



Podríamos asegurar el uso de claves públicas/privadas y deshabilitar el acceso con contraseña en el archivo /etc/ssh/sshd_config.

¿Es necesario?
Sí, si se busca una alta seguridad en las conexiones SSH ya que reemplazar las contraseñas por claves públicas/privadas dificulta ataques de fuerza bruta y aumenta la seguridad.

En nuestro caso es simplemente un servidor de pruebas para una práctica, por lo que no lo consideramos necesario.

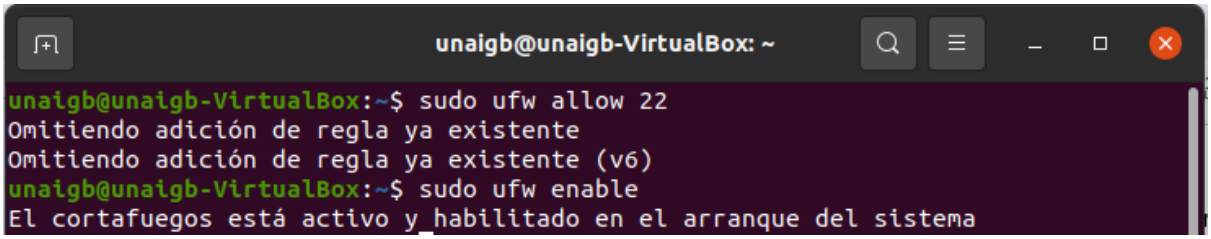
FTP

```
unaigb@unaigb-VirtualBox:~$ sudo apt install vsftpd
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias
Leyendo la información de estado... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no
son necesarios.
```

Una vez instalado, lo habilitaremos y lo iniciaremos:

```
unaigb@unaigb-VirtualBox:~$ sudo systemctl enable vsftpd
Synchronizing state of vsftpd.service with SysV service script with /lib/systemd
/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable vsftpd
unaigb@unaigb-VirtualBox:~$ sudo systemctl start vsftpd
```

Además habilitaremos el acceso desde el puerto 22:



Ahora podríamos habilitar SSL editando /etc/vsftpd.conf y especificando el certificado y clave en caso de querer activar conexiones encriptadas.

¿Es necesario?
Sí, si se transfieren datos sensibles. El FTP sin cifrar (modo plano), transmite las credenciales y los datos sin protección, lo que puede ser capturado por atacantes, por lo que usar opciones como FTPS (FTP con SSL/TLS) es esencial en redes inseguras o públicas.

Como hemos dicho antes, nuestro caso es simplemente un servidor de pruebas para una práctica, por lo que no lo consideramos necesario.

HTTP

Procedemos a instalar apache para este servicio:

```
unaigb@unaigb-VirtualBox:~$ sudo apt install apache2
[sudo] contraseña para unaigb:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias
Leyendo la información de estado... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no
son necesarios.
```

Y ahora lanzamos el servidor web:

```
unaigb@unaigb-VirtualBox:~$ sudo a2enmod ssl
Considering dependency setenvif for ssl:
Module setenvif already enabled
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create s
elf-signed certificates.
To activate the new configuration, you need to run:
  systemctl restart apache2
unaigb@unaigb-VirtualBox:~$ sudo systemctl restart apache2
```

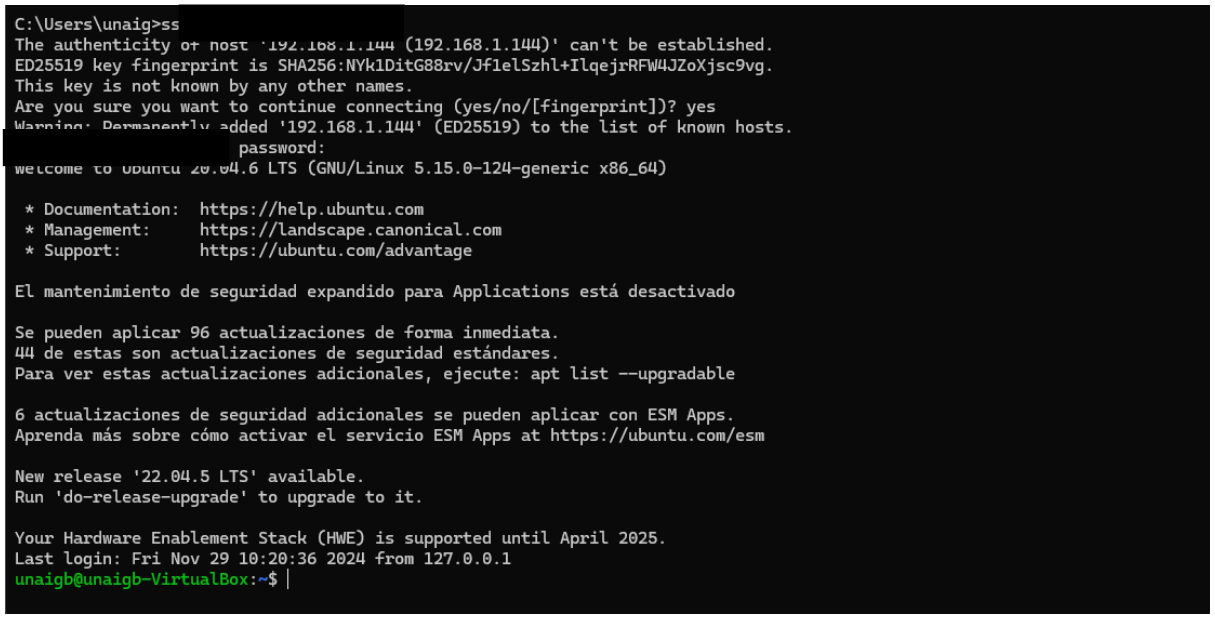
¿Es necesario?

Sí, sí ofreciéramos un servicio web que fuese accesible desde Internet. Los navegadores modernos exigen HTTPS para proteger las conexiones contra ataques como el "Man-in-the-Middle". Además, un certificado válido evita alertas en los navegadores de los usuarios.

En nuestro caso, simplemente accederemos a él desde otro equipo en la misma red, por lo que no es necesario.

Pruebas

Una vez hecho todo esto, podremos acceder al servidor desde otros equipos de la misma red local, en nuestro caso, teníamos el servidor en una máquina virtual en un portátil y accedimos desde otro equipo de sobremesa con Windows:

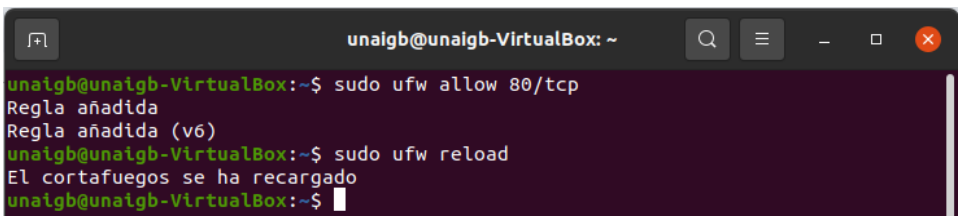


Y podríamos subir un archivo fácilmente.



En este punto nos encontramos con el problema de que FTP no reconocía el host, por lo que decidimos usar su sucesor SFTP que usa directamente el canal seguro para el paso de archivos.

Para probar el acceso al servidor HTTP, primero deberemos permitir el acceso desde el puerto 80 mediante TCP:



Y ya podremos acceder:

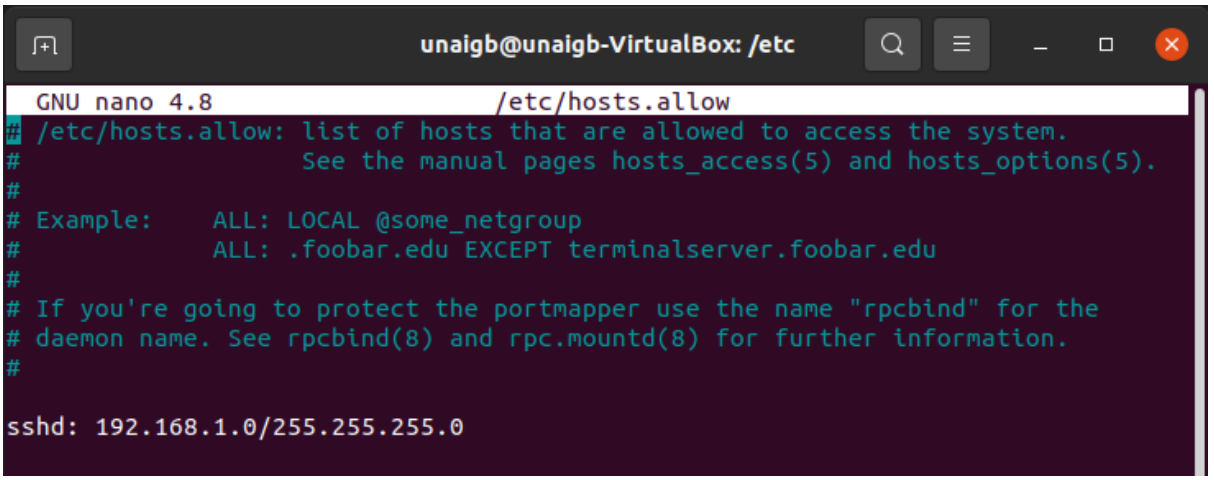


1.2. TCP Wrappers

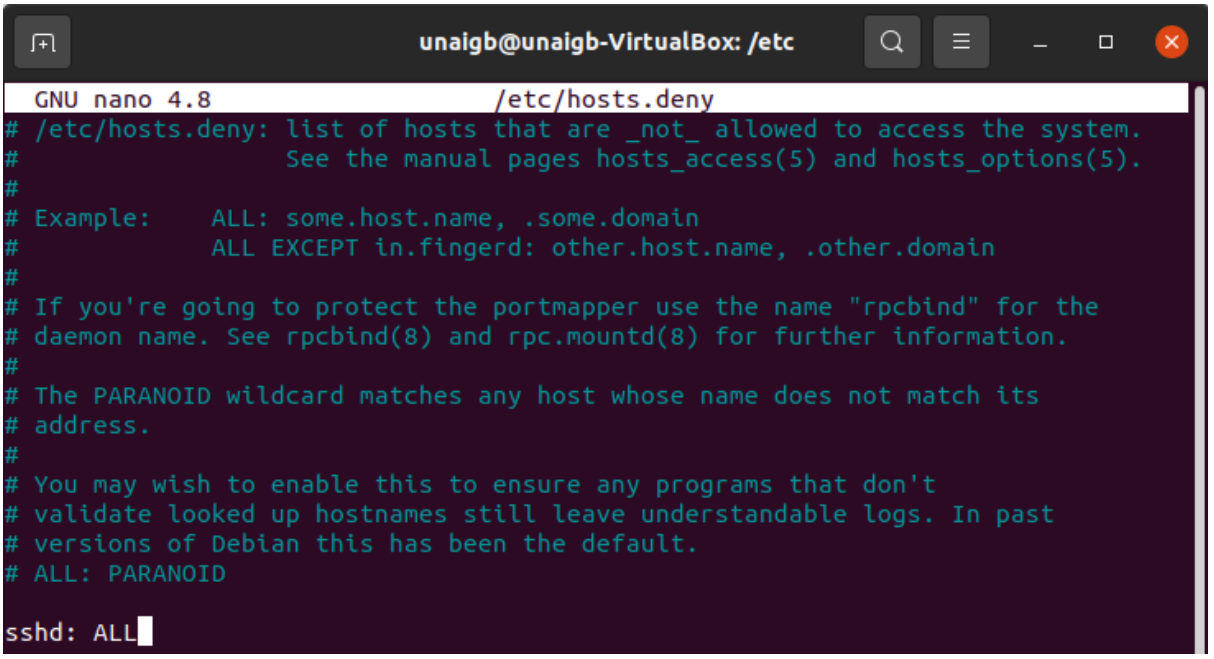
El objetivo es filtrar el acceso a los servicios web, y se hará para el filtrado de tráfico a dichos servicios para usuarios por direcciones IP y por nombres de dominio.
TCP Wrappers controla el acceso a servicios mediante los archivos /etc/hosts.allow y /etc/hosts.deny por lo que haremos algunas modificaciones en ellos:

- **Permitir solo acceso a SSH desde la subred local (192.168.1.0/24):**

  En /etc/hosts.allow:



En /etc/hosts.deny:



- **Permitir acceso al servidor FTP solo desde un equipo específico (192.168.1.100):**

  En /etc/hosts.allow:

  vsftpd: 192.168.1.100

  En /etc/hosts.deny:

  vsftpd: ALL

- **Permitir acceso al servidor web HTTPS desde dominios específicos:**

  En /etc/hosts.allow:

  apache2: .example.com

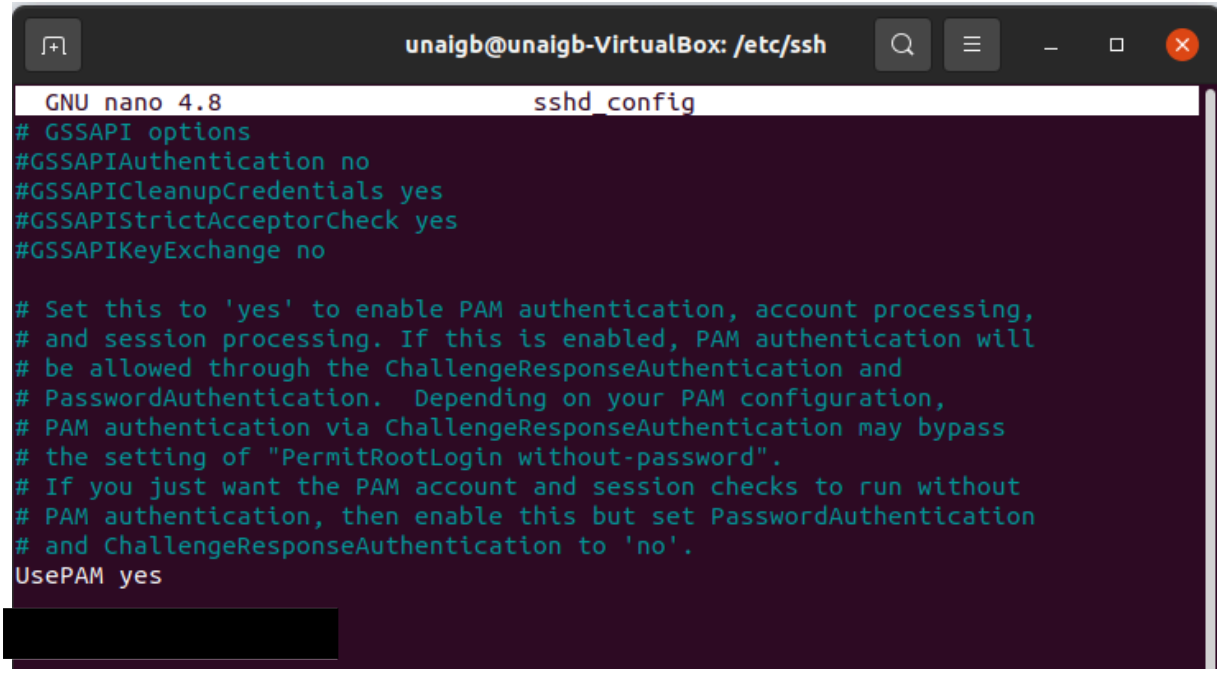
  En /etc/hosts.deny:

  apache2: ALL

Escenarios de prueba:

- Intentar acceder a SSH desde una IP no permitida.
- Comprobar el acceso a FTP desde una máquina específica.
- Probar accesos HTTPS desde dominios permitidos y denegados.

Para la configuración por IP vamos a comenzar configurando el archivo sshd_config:

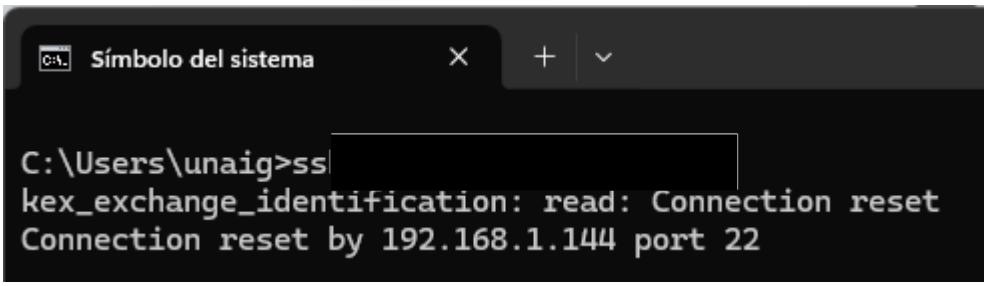


Configurando las conexiones para que solo se pueda acceder al usuario indicado con la ip indicada.

Como pequeña prueba rápida, cambiaremos en hosts.allow el valor de sshd al siguiente:

```
sshd: 192.168.1.209
```

Esto hará que solo el equipo con dicha IP pueda acceder al servidor, por lo que si intentamos acceder, veremos lo siguiente:



Quedando claro que no podemos acceder con esta configuración al servidor.

1.3. Comparación: TCP Wrappers vs IPTables.

Ambas son herramientas que se utilizan para controlar y filtrar el tráfico de red en sistemas basados en Linux.

Iptables: Es un cortafuegos basado en reglas que opera a nivel de kernel, siendo una herramienta más integral y poderosa que TCP wrappers. Iptables se utiliza para filtrar,

redirigir y manipular paquetes de red en el nivel del kernel, pudiendo realizar filtrado por dirección IP, puerto, protocolo y más. Las reglas de iptables se aplican a nivel de kernel antes de que los paquetes lleguen a los servicios de red específicos, esto significa que iptables puede filtrar paquetes antes de que lleguen a la aplicación destinataria. Las reglas tienen una sintaxis específica y permiten un control granular sobre el tráfico de red. Puedes especificar condiciones detalladas y acciones específicas para diferentes tipos de tráfico. Iptables es compatible con una amplia variedad de protocolos y servicios y se puede utilizar para aplicar políticas de seguridad a nivel de sistema.

TCP wrappers: Es una herramienta de filtrado de acceso que opera a nivel de las aplicaciones de red. Se enfoca específicamente en controlar el acceso a servicios de red a través de los archivos “/etc/hosts.allow” y “/etc/hosts.deny”. TCP wrappers se utiliza principalmente para permitir o denegar el acceso a servicios específicos basados en la dirección IP del cliente o el nombre de dominio. Se centra en el control de acceso a servicios de red específicos y no manipula paquetes a nivel de kernel. TCP wrappers opera a nivel de aplicación. Las decisiones de acceso se toman después de que los paquetes de red han llegado al servicio de red, como SSH o SFTP.
Las reglas son más simples y se centran en permitir o denegar el acceso a servicios específicos basados en direcciones IP o nombres de dominio.
Como comparación más general se puede decir que iptables ofrece un control más profundo y granular sobre el tráfico de red en todo el sistema mientras que TCP wrappers se enfoca en el control de acceso a servicios específicos.
Iptables es más adecuado para implementar políticas de seguridad a nivel de sistema y para manipular paquetes de red, mientras que TCP wrappers es más adecuado para controlar el acceso a servicios de red específicos.
Ambas herramientas pueden utilizarse de manera conjunta para proporcionar una capa adicional de seguridad. Iptables puede proteger contra ataques a nivel de sistema mientras que TCP wrappers puede controlar el acceso a servicios específicos.

2. Análisis de servicios activos

2.1. Análisis de servicios activos

Un análisis de servicios activos implica identificar los programas o procesos que están escuchando conexiones en la máquina y los puertos que tienen abiertos. En nuestro caso, y como requiere la práctica, lo vamos a realizar tanto en **Linux** como en **Windows** con herramientas específicas.

2.1.1. Usar netstat para analizar servicios activos

En Linux:

Abrimos la terminal e introducimos el siguiente comando:

```
unaigb@unaigb-VirtualBox:/etc/ssh$ sudo netstat -tuln
[sudo] contraseña para unaigb:
Conexiones activas de Internet (solo servidores)
Proto  Recib Enviad Dirección local         Dirección remota       Estado
tcp        0      0 127.0.0.1:631           0.0.0.0:*               ESCUCHAR
tcp        0      0 0.0.0.0:22              0.0.0.0:*               ESCUCHAR
tcp        0      0 127.0.0.53:53           0.0.0.0:*               ESCUCHAR
tcp6       0      0 :::80                   :::*                    ESCUCHAR
tcp6       0      0 :::22                   :::*                    ESCUCHAR
tcp6       0      0 :::21                   :::*                    ESCUCHAR
tcp6       0      0 :::443                  :::*                    ESCUCHAR
tcp6       0      0 ::1:631                 :::*                    ESCUCHAR
udp        0      0 0.0.0.0:55491           0.0.0.0:*
udp        0      0 0.0.0.0:59613           0.0.0.0:*
udp        0      0 0.0.0.0:43763           0.0.0.0:*
udp        0      0 0.0.0.0:43790           0.0.0.0:*
udp        0      0 0.0.0.0:35742           0.0.0.0:*
udp        0      0 0.0.0.0:52341           0.0.0.0:*
udp        0      0 0.0.0.0:60850           0.0.0.0:*
udp        0      0 0.0.0.0:40565           0.0.0.0:*
udp        0      0 0.0.0.0:40600           0.0.0.0:*
udp        0      0 0.0.0.0:40645           0.0.0.0:*
udp        0      0 127.0.0.53:53           0.0.0.0:*
udp        0      0 0.0.0.0:45146           0.0.0.0:*
udp        0      0 0.0.0.0:37045           0.0.0.0:*
udp        0      0 0.0.0.0:49489           0.0.0.0:*
udp        0      0 0.0.0.0:45516           0.0.0.0:*
udp        0      0 0.0.0.0:33420           0.0.0.0:*
udp        0      0 0.0.0.0:41649           0.0.0.0:*
udp        0      0 0.0.0.0:49877           0.0.0.0:*
udp        0      0 0.0.0.0:41764           0.0.0.0:*
udp        0      0 0.0.0.0:42024           0.0.0.0:*
udp        0      0 0.0.0.0:58540           0.0.0.0:*
udp        0      0 0.0.0.0:5353            0.0.0.0:*
udp        0      0 0.0.0.0:50443           0.0.0.0:*
udp        0      0 0.0.0.0:58783           0.0.0.0:*
udp        0      0 0.0.0.0:54707           0.0.0.0:*
udp        0      0 0.0.0.0:34330           0.0.0.0:*
udp        0      0 0.0.0.0:38617           0.0.0.0:*
udp        0      0 0.0.0.0:34537           0.0.0.0:*
udp6       0      0 :::56109                :::*
udp6       0      0 :::5353                 :::*
```

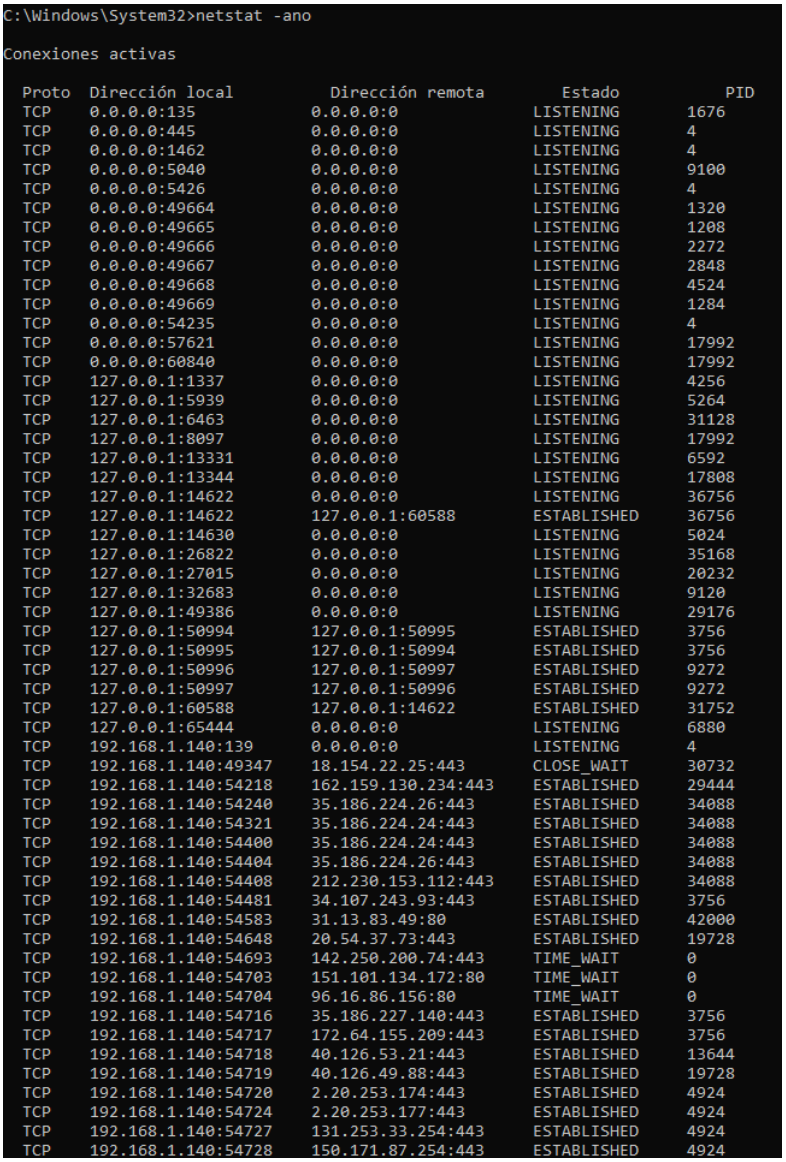
Esto mostrará una lista de:

- Protocolos (tcp o udp).
- Puertos en escucha.
- Dirección IP y puerto asociado.
- Estado de la conexión (solo para tcp).

Según la salida obtenida, lo más interesante es comprobar como en la columna de dirección local, observamos los puertos de escucha 80 y 22, siendo estos para HTTP y SSH respectivamente.

En Windows:

Abriremos la terminal como administrador y ejecutaremos el siguiente comando:



```
C:\Windows\System32>netstat -ano

Conexiones activas

  Proto  Dirección local          Dirección remota        Estado           PID
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING       1676
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:1462           0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:5040           0.0.0.0:0              LISTENING       9100
  TCP    0.0.0.0:5426           0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:49664          0.0.0.0:0              LISTENING       1320
  TCP    0.0.0.0:49665          0.0.0.0:0              LISTENING       1208
  TCP    0.0.0.0:49666          0.0.0.0:0              LISTENING       2272
  TCP    0.0.0.0:49667          0.0.0.0:0              LISTENING       2848
  TCP    0.0.0.0:49668          0.0.0.0:0              LISTENING       4524
  TCP    0.0.0.0:49669          0.0.0.0:0              LISTENING       1284
  TCP    0.0.0.0:54235          0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:57621          0.0.0.0:0              LISTENING       17992
  TCP    0.0.0.0:60840          0.0.0.0:0              LISTENING       17992
  TCP    127.0.0.1:1337         0.0.0.0:0              LISTENING       4256
  TCP    127.0.0.1:5939         0.0.0.0:0              LISTENING       5264
  TCP    127.0.0.1:6463         0.0.0.0:0              LISTENING       31128
  TCP    127.0.0.1:8097         0.0.0.0:0              LISTENING       17992
  TCP    127.0.0.1:13331        0.0.0.0:0              LISTENING       6592
  TCP    127.0.0.1:13344        0.0.0.0:0              LISTENING       17808
  TCP    127.0.0.1:14622        0.0.0.0:0              LISTENING       36756
  TCP    127.0.0.1:14622        127.0.0.1:60588        ESTABLISHED     36756
  TCP    127.0.0.1:14630        0.0.0.0:0              LISTENING       5024
  TCP    127.0.0.1:26822        0.0.0.0:0              LISTENING       35168
  TCP    127.0.0.1:27015        0.0.0.0:0              LISTENING       20232
  TCP    127.0.0.1:32683        0.0.0.0:0              LISTENING       9120
  TCP    127.0.0.1:49386        0.0.0.0:0              LISTENING       29176
  TCP    127.0.0.1:50994        127.0.0.1:50995        ESTABLISHED     3756
  TCP    127.0.0.1:50995        127.0.0.1:50994        ESTABLISHED     3756
  TCP    127.0.0.1:50996        127.0.0.1:50997        ESTABLISHED     9272
  TCP    127.0.0.1:50997        127.0.0.1:50996        ESTABLISHED     9272
  TCP    127.0.0.1:60588        127.0.0.1:14622        ESTABLISHED     31752
  TCP    127.0.0.1:65444        0.0.0.0:0              LISTENING       6880
  TCP    192.168.1.140:139      0.0.0.0:0              LISTENING       4
  TCP    192.168.1.140:49347    18.154.22.25:443       CLOSE_WAIT      30732
  TCP    192.168.1.140:54218    162.159.130.234:443    ESTABLISHED     29444
  TCP    192.168.1.140:54240    35.186.224.26:443      ESTABLISHED     34088
  TCP    192.168.1.140:54321    35.186.224.24:443      ESTABLISHED     34088
  TCP    192.168.1.140:54400    35.186.224.24:443      ESTABLISHED     34088
  TCP    192.168.1.140:54404    35.186.224.26:443      ESTABLISHED     34088
  TCP    192.168.1.140:54408    212.230.153.112:443    ESTABLISHED     34088
  TCP    192.168.1.140:54481    34.107.243.93:443      ESTABLISHED     3756
  TCP    192.168.1.140:54583    31.13.83.49:80         ESTABLISHED     42000
  TCP    192.168.1.140:54648    20.54.37.73:443        ESTABLISHED     19728
  TCP    192.168.1.140:54693    142.250.200.74:443     TIME_WAIT       0
  TCP    192.168.1.140:54703    151.101.134.172:80     TIME_WAIT       0
  TCP    192.168.1.140:54704    96.16.86.156:80        TIME_WAIT       0
  TCP    192.168.1.140:54716    35.186.227.140:443     ESTABLISHED     3756
  TCP    192.168.1.140:54717    172.64.155.209:443     ESTABLISHED     3756
  TCP    192.168.1.140:54718    40.126.53.21:443       ESTABLISHED     13644
  TCP    192.168.1.140:54719    40.126.49.88:443       ESTABLISHED     19728
  TCP    192.168.1.140:54720    2.20.253.174:443       ESTABLISHED     4924
  TCP    192.168.1.140:54724    2.20.253.177:443       ESTABLISHED     4924
  TCP    192.168.1.140:54727    131.253.33.254:443     ESTABLISHED     4924
  TCP    192.168.1.140:54728    150.171.87.254:443     ESTABLISHED     4924
```

Esto mostrará:

- Protocolos (tcp o udp).
- Puertos en escucha.
- Estado de las conexiones (LISTENING, ESTABLISHED).

- PID del proceso asociado.

En este caso, lo hemos comprobado en el Windows desde el que accedimos al servidor ssh en el apartado anterior, por lo que podemos observar como no encontramos los puertos 80 y 22 como en el caso del Linux.

Si quisiéramos identificar qué servicio está usando un puerto específico, podríamos usar el siguiente comando: "*tasklist* | *findstr <PID>*"

2.1.2. Usar nmap para analizar puertos abiertos

La herramienta "nmap" es muy poderosa para analizar los puertos abiertos en una máquina, además de útil para comprobar la configuración desde una máquina remota. Para ello, instalaremos nmap en nuestro equipo.

```
C:\Users\unaig>nmap --version
Nmap version 7.95 ( https://nmap.org )
Platform: i686-pc-windows-windows
Compiled with: nmap-liblua-5.4.6 openssl-3.0.13 nmap-libssh2-1.11.0 nmap-libz-1.3.1 nmap-libpcre2-10.43 Npcap-1.79 nmap-
libdnet-1.12 ipv6
Compiled without:
Available nsock engines: iocp poll select
```

Una vez instalado, podemos hacer varias pruebas y escaneos:

- **Escaneo básico:**

  Desde otra máquina en la red, esto listará los puertos abiertos y los servicios asociados.

```
C:\Users\unaig>nmap 192.168.1.144
Starting Nmap 7.95 ( https://nmap.org ) at 2024-12-02 21:36 Hora estßndar romance
Nmap scan report for unaigb-virtualbox (192.168.1.144)
Host is up (0.039s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
MAC Address: 70:66:55:6C:2A:4F (AzureWave Technology)

Nmap done: 1 IP address (1 host up) scanned in 10.44 seconds
```

  Y efectivamente, nos reconoce el servidor a la perfección.

- **Escanear puertos específicos:**

  Si solo queremos comprobar servicios concretos como SSH, FTP o HTTP.

```
C:\Users\unaig>nmap -p 22,21,80,443 192.168.1.144
Starting Nmap 7.95 ( https://nmap.org ) at 2024-12-02 21:38 Hora estßndar romance
Nmap scan report for unaigb-virtualbox (192.168.1.144)
Host is up (0.072s latency).

PORT    STATE    SERVICE
21/tcp  filtered ftp
22/tcp  open     ssh
80/tcp  open     http
443/tcp filtered https
MAC Address: 70:66:55:6C:2A:4F (AzureWave Technology)

Nmap done: 1 IP address (1 host up) scanned in 2.13 seconds
```

- **Usar un escaneo más avanzado:**

  Ya que podremos identificar las versiones de los servicios

```
C:\Users\unaig>nmap -sV 192.168.1.144
Starting Nmap 7.95 ( https://nmap.org ) at 2024-12-02 21:39 Hora estßndar romance
Nmap scan report for unaigb-virtualbox (192.168.1.144)
Host is up (0.015s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.11 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Apache httpd 2.4.41 ((Ubuntu))
MAC Address: 70:66:55:6C:2A:4F (AzureWave Technology)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.83 seconds
```

2.2. Herramientas adicionales de análisis de puertos

2.2.1. Herramientas alternativas a nmap

Algunas de las herramientas que sirvan como alternativa de nmap que hemos encontrado son las siguientes:

Zenmap:

- Es la interfaz gráfica oficial de nmap.
- Permite realizar escaneos de red detallados y visualizar los resultados de forma más amigable.
- Compatible con Linux, Windows y macOS.

Angry IP Scanner:

- Interfaz gráfica simple para analizar IPs y puertos en una red.
- Muy útil para análisis rápidos.
- Compatible con Linux, Windows y macOS.

Netcat (nc):

- Herramienta de línea de comandos para interactuar con puertos de red.
- Ideal para pruebas simples de conectividad.
- Ejemplo para comprobar si el puerto 80 está accesible:

OpenVAS:

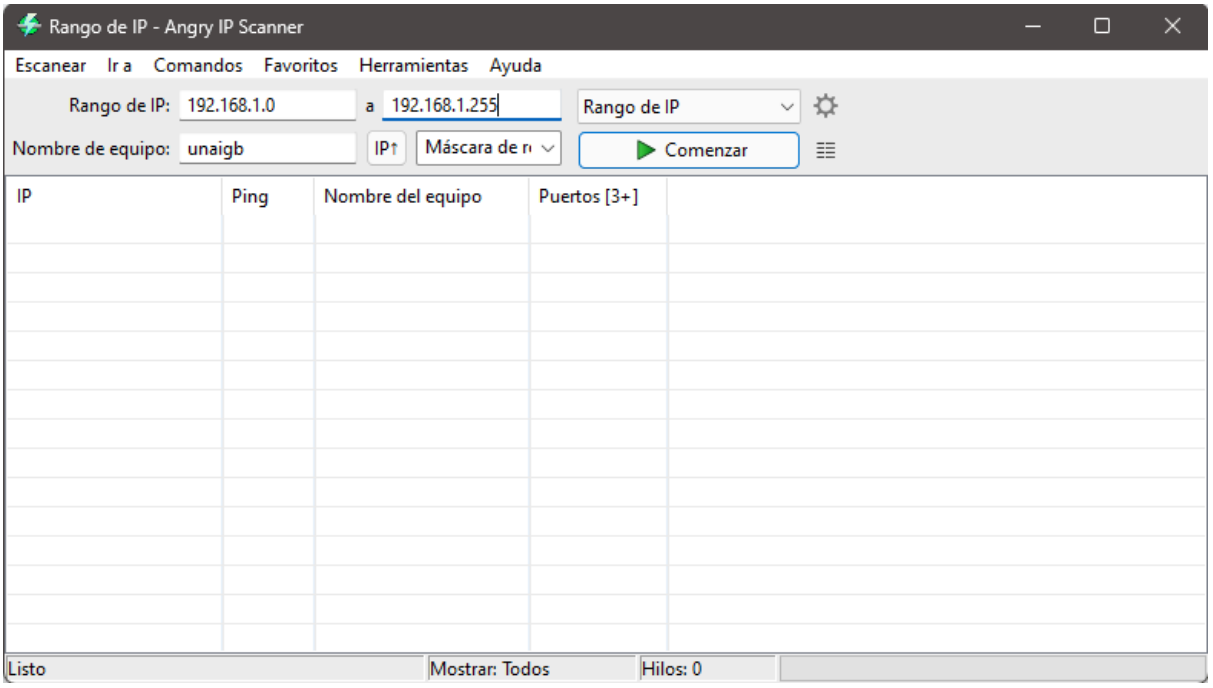
- Escáner de vulnerabilidades que también identifica puertos abiertos.
- Requiere configuración avanzada, pero es ideal para análisis de seguridad en profundidad.

2.2.2. Ejemplo práctico: Uso de Angry IP Scanner

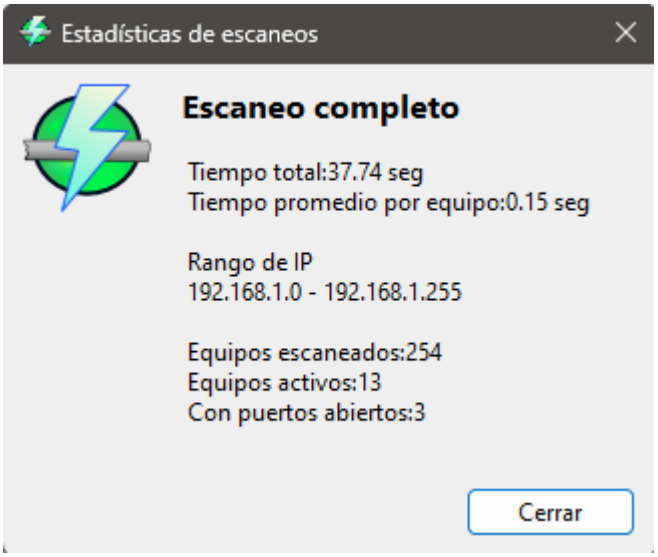
Para llevar a cabo las pruebas, hemos elegido la herramienta "*Angry IP Scanner*", por lo que primero instalaremos la aplicación desde el siguiente enlace:

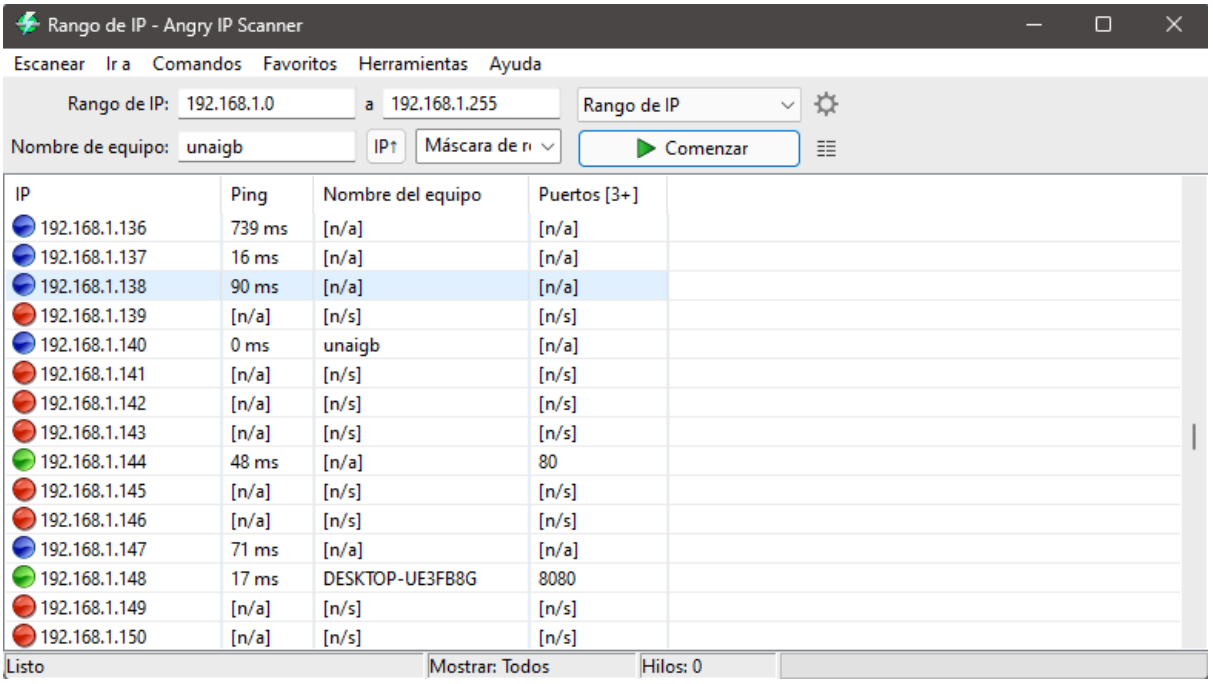
https://angryip.org/

Una vez instalado y abierto, veremos la siguiente interfaz:



Donde en la parte superior observamos el rango de IP's que podemos escanear, en este caso la local, por lo que iniciamos el escaneo. El resultado es el siguiente:





Donde podremos observar una lista de dispositivos en la red con puertos abiertos, además de que podremos identificar rápidamente qué servicios están disponibles en el servidor.